

YouTube :

Functional requirements:

User should be able to view/download any video based on his preference,

User's feed should be updated as the user's activity,

User should be able to upload a file >50GB

Non-functional requirements:

Data-partitioning,

load balancing,.

fault tolerance,

vertical scalability,

horizontal scalability,

low latency,

high throughput,

CAP theorem (Here eventual consistency is fine , but here we need 99.99% uptime

i.e Availability>>Consistency)

Core Entities:

user ,

video,

history

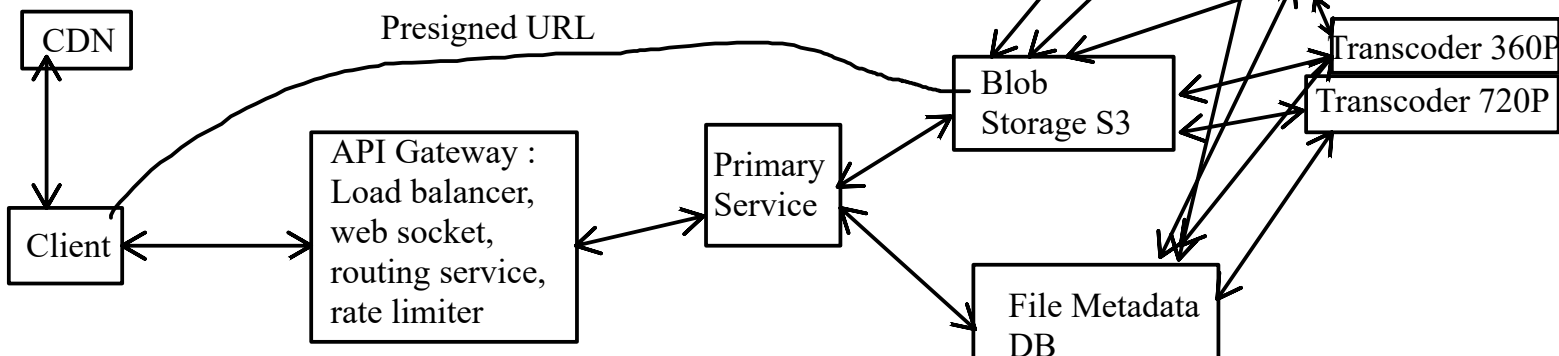
APIs :

POST:/upload

```
{
  userID,
  {filemetadata}
}
```

GET:/search/{videoID}

HLD :



- 1) we need to Chunk 50GB file into multiple chunks
- 2) we need to fingerprint or hash each piece of these chunks separately
- 3) we need to upload each piece of these chunks separately
- 4) update completed status for each piece of these chunks separately